

中山大學
SUN YAT-SEN UNIVERSITY

数据库期末大作业

智慧物流系统设计与实现

组员姓名: 欧阳易芃、马福泉、刘昊

组员学号: 23336188/23336179/23336150

专　　业: 计算机科学与技术

课程教师: 赖韩江

起始日期: 2025 年 12 月 27 日

结束日期: 2026 年 1 月 16 日

实验地点: 超算 307/宿舍

目录

1 小组成员与分工总览 3

2 概念结构设计 3

2.1 实体与属性定义 . . . 3
2.2 E-R 图 . . . 4

3 逻辑结构设计 5

3.1 E-R 图转换为关系模式 . . . 5
3.2 规范化分析 . . . 6

4 物理结构与高级对象设计 6

4.1 表结构定义 . . . 6
4.1.1 创建 DistributionCenter 表并初始化 . . . 6
4.1.2 创建 Fleet 表并初始化 . . . 7
4.1.3 创建 Vehicle 表并初始化 . . . 7
4.1.4 创建 Driver 表并初始化 . . . 8
4.1.5 创建 Orders 表并初始化 . . . 8
4.1.6 创建 Abnormality 表并初始化 . . . 8
4.1.7 创建 History_Log 表 . . . 9
4.2 Trigger 的详细设计与代码实现 . . . 9
4.2.1 车辆分配后自动变更状态 . . . 9
4.2.2 运单签收后车辆自动闲置 . . . 9
4.2.3 司机驾照变更自动审计 . . . 10
4.3 存储过程与视图的设计 . . . 10
4.4 索引的设置策略 . . . 12

5 系统实现与测试 14

5.1 开发环境说明 . . . 14
5.2 关键功能截图 . . . 15
5.2.1 录入信息后校验 . . . 15
5.2.2 分配车辆后状态变更 . . . 15
5.2.3 运单签收后车辆闲置 . . . 16
5.2.4 司机驾照变更审计 . . . 17
5.2.5 异常事件录入 . . . 17
5.3 前端技术选型与关键代码 . . . 17
5.4 业务逻辑扩展与实现 . . . 20
5.4.1 车队资源实时监控 . . . 20
5.4.2 司机绩效多维度追踪 . . . 21
5.4.3 车队月度安全与效率报表 . . . 22
5.4.4 前端交互优化与无障碍支持 . . . 22
5.5 技术难点及解决方案 . . . 22

6 总结 23

7 附录/附件 **23**
7.1 演示 demo 及视频链接 23
7.2 主要 SQL 脚本 23
7.2.1 一键初始化脚本（init_DB.sql） 23
7.2.2 修复与高级对象脚本（patch.sql） 26

1 小组成员与分工总览

姓名	学号	任务分工
欧阳易芃	23336188	数据库设计（33%）、触发器/存储过程（33%）、后端接口（39%）、单元测试（33%）、简单前端（35%）、实验报告（30%）、视频讲解（70%）
马福泉	23336179	数据库设计（33%）、触发器/存储过程（33%）、后端接口（49%）、单元测试（33%）、简单前端（25%）、实验报告（30%）、视频讲解（15%）
刘昊	23336150	数据库设计（33%）、触发器/存储过程（33%）、后端接口（12%）、单元测试（33%）、简单前端（40%）、实验报告（30%）、视频讲解（15%）

2 概念结构设计

2.1 实体与属性定义

1. **DistributionCenter（配送中心）：**
   - **性质：**强实体。
   - **属性：**选取 `Center_ID` 作为主键（Primary Key，下划线标出）唯一标识一个配送中心。Name（中心名称）、Location（地址）为普通单值属性。
2. **Fleet（车队）：**
   - **性质：**强实体。
   - **属性：**选取 `Fleet_ID` 作为主键（Primary Key，下划线标出）唯一标识一个车队。FleetName（车队名称）、SupervisorName（调度主管）为普通单值属性。Center_ID 为外键，指向所属配送中心。
   - **设计说明：**每个车队只属于一个配送中心，一个配送中心可有多个车队（1:n）。
3. **Vehicle（车辆）：**
   - **性质：**强实体。
   - **属性：**选取 `Vehicle_ID` 作为主键（Primary Key，下划线标出）唯一标识一辆车。PlateNumber（唯一车牌）、MaxWeight（最大载重）、MaxVolume（最大容积）、CurrentStatus（状态，枚举：空闲/运输中/维修中/异常）为普通单值属性。Fleet_ID 为外键，指向所属车队。
4. **Driver（司机）：**
   - **性质：**强实体。
   - **属性：**选取 `Driver_ID` 作为主键（Primary Key，下划线标出）唯一标识一名司机。Name（姓名）、LicenseGrade（驾照等级，枚举 A1/A2/B1/B2/C1）、Phone（联系方式）为普通单值属性。Fleet_ID 为外键，指向所属车队。
5. **Orders（运单）：**

- **性质：**强实体。
- **属性：**选取 `Order_ID` 作为主键（Primary Key，下划线标出）唯一标识一张运单。Weight（货物重量）、Volume（货物体积）、Destination（目的地）、Status（状态，枚举：待分配/运输中/已完成）、OrderTime（下单时间）为普通单值属性。Vehicle_ID（关联车辆）、Driver_ID（关联司机）为外键，可为空，指向分配资源。

6. **Abnormality（异常记录）：**

- **性质：**强实体。
- **属性：**选取 `Abnormality_ID` 作为主键（Primary Key，下划线标出）唯一标识一条异常记录。EventType（异常类型）、OccurTime（发生时间）、FineAmount（罚款金额）、IsHandled（是否已处理）为普通单值属性。Driver_ID、Vehicle_ID 为外键，分别指向关联司机和车辆。

7. **History_Log（审计日志）：**

- **性质：**强实体。
- **属性：**选取 `Log_ID` 作为主键（Primary Key，下划线标出）唯一标识一条日志。TableName（表名）、Record_ID（被操作记录 ID）、OperationType（操作类型）、OldValue（旧值）、NewValue（新值）、LogTime（记录时间）为普通单值属性。

2.2 E-R 图

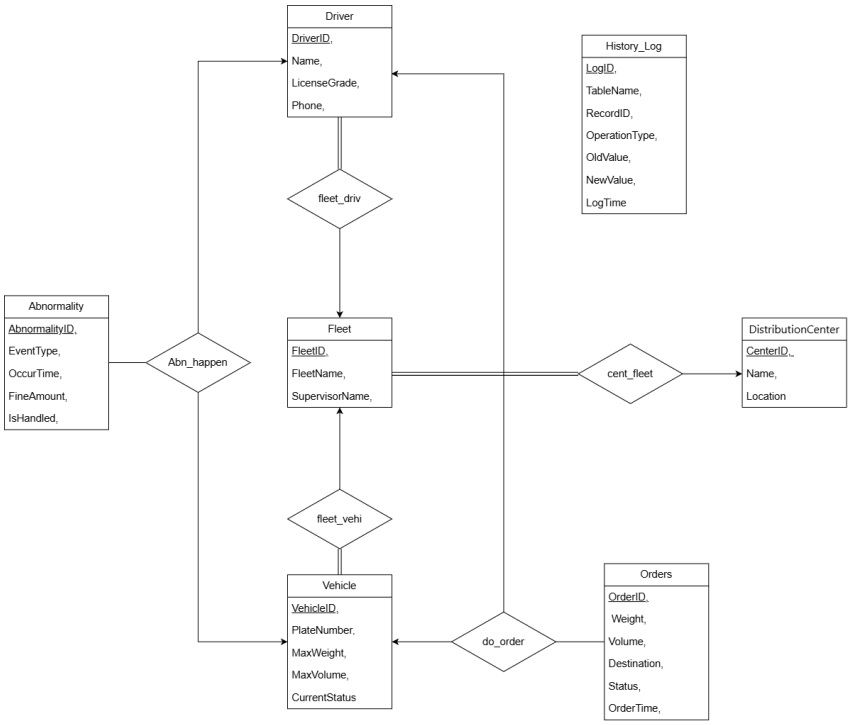


图 1: E-R 图

同时，我也在 DataGrip 中自动生成了一张关系图，并手动调整（它并不是符合课本要求的图）：

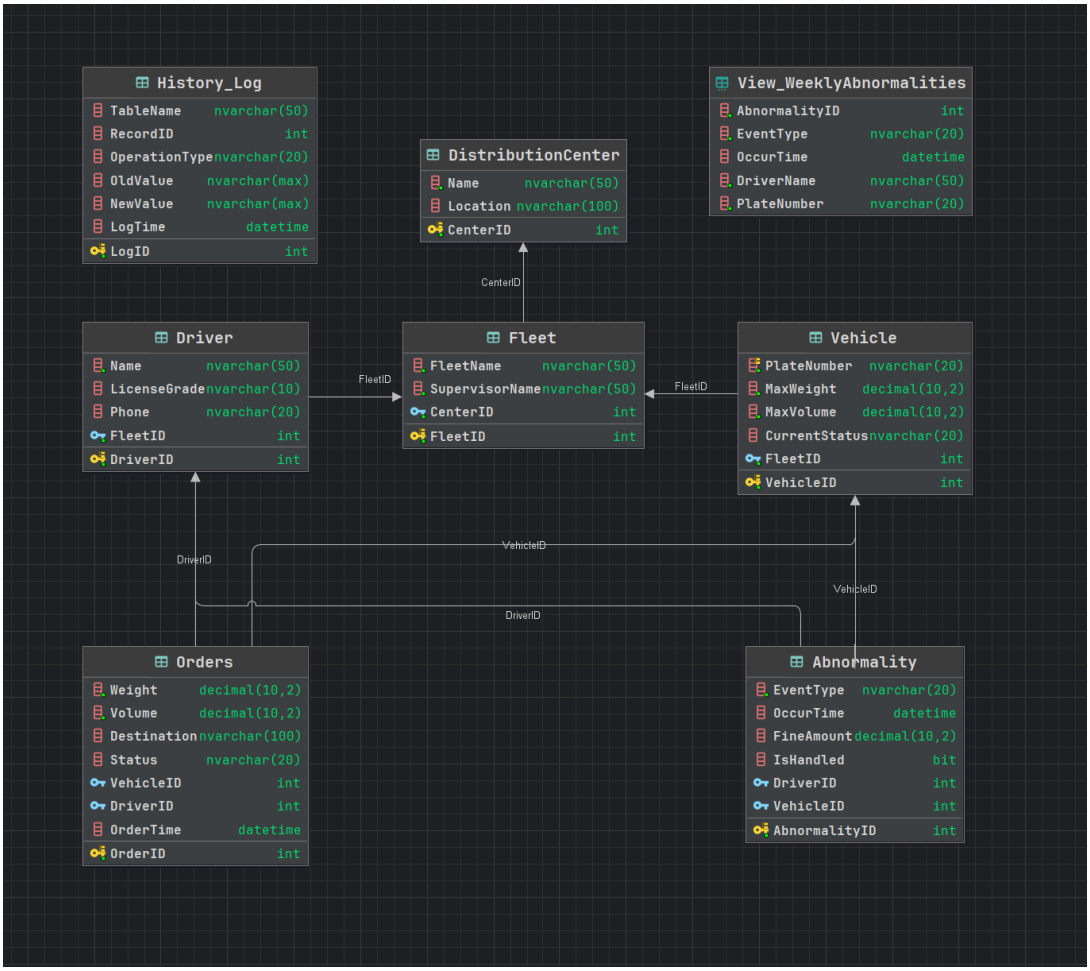


图 2: 关系图（DataGrip 自动生成）

3 逻辑结构设计

3.1 E-R 图转换为关系模式

根据 E-R 图，系统各实体转换为如下关系模式：

- **DistributionCenter** (CenterID, Name, Location)
- **Fleet** (FleetID, FleetName, SupervisorName, CenterID)
- **Vehicle** (VehicleID, PlateNumber, MaxWeight, MaxVolume, CurrentStatus, FleetID)
- **Driver** (DriverID, Name, LicenseGrade, Phone, FleetID)
- **Orders** (OrderID, Weight, Volume, Destination, Status, VehicleID, DriverID, OrderTime)
- **Abnormality** (AbnormalityID, EventType, OccurTime, FineAmount, IsHandled, DriverID, VehicleID)
- **History_Log** (LogID, TableName, RecordID, OperationType, OldValue, NewValue, LogTime)

3.2 规范化分析

本系统所有表均已达到第三范式（3NF），具体分析如下：

- **1NF（第一范式）**：所有表的每个字段都不可再分，均为原子属性，无重复组。
- **2NF（第二范式）**：所有表均有主键，且不存在部分依赖（无复合主键，所有非主属性完全依赖主键）。
- **3NF（第三范式）**：所有非主属性均直接依赖主键，不依赖于其他非主属性。例如：
  - **DistributionCenter**：主键为 CenterID，所有非主属性（Name, Location）均完全且仅依赖于 CenterID，无传递依赖。
  - **Fleet**：主键为 FleetID，所有非主属性（FleetName, SupervisorName, CenterID）均完全依赖于 FleetID。CenterID 是外键，依赖于 DistributionCenter 的主键，不是本表的非主属性，无传递依赖。
  - **Vehicle**：主键为 VehicleID，所有非主属性（PlateNumber, MaxWeight, MaxVolume, CurrentStatus, FleetID）均完全依赖于 VehicleID。FleetID 是外键，依赖于 Fleet 的主键，无传递依赖。
  - **Driver**：主键为 DriverID，所有非主属性（Name, LicenseGrade, Phone, FleetID）均完全依赖于 DriverID。FleetID 是外键，依赖于 Fleet 的主键，无传递依赖。
  - **Orders**：主键为 OrderID，所有非主属性（Weight, Volume, Destination, Status, VehicleID, DriverID, OrderTime）均完全依赖于 OrderID。VehicleID、DriverID 是外键，依赖于 Vehicle、Driver 的主键，无传递依赖。
  - **Abnormality**：主键为 AbnormalityID，所有非主属性（EventType, OccurTime, FineAmount, IsHandled, DriverID, VehicleID）均完全依赖于 AbnormalityID。DriverID、VehicleID 均为外键，依赖于对应主表主键，无传递依赖。
  - **History_Log**：主键为 LogID，所有非主属性（TableName, RecordID, OperationType, OldValue, NewValue, LogTime）均完全依赖于 LogID，无传递依赖。

  综上，所有表的每个非主属性都直接且完全依赖于主键，不依赖于其他非主属性，也不存在传递依赖，均满足第三范式（3NF）。
- **无冗余依赖**：所有外键均为单向指向，未出现传递依赖或冗余字段。

因此，所有表结构均已达到 3NF，满足规范化要求。

4 物理结构与高级对象设计

4.1 表结构定义

4.1.1 创建 DistributionCenter 表并初始化

```sql
CREATE TABLE DistributionCenter (
    CenterID INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(50) NOT NULL,
    Location NVARCHAR(100)
);
INSERT INTO DistributionCenter (Name, Location) VALUES ('北京东区中心', '北京市朝阳区');
```

4.1.2 创建 Fleet 表并初始化

```sql
CREATE TABLE Fleet (
    FleetID INT PRIMARY KEY IDENTITY(1,1),
    FleetName NVARCHAR(50) NOT NULL,
    SupervisorName NVARCHAR(50) NOT NULL,
    CenterID INT NOT NULL,
    FOREIGN KEY (CenterID) REFERENCES DistributionCenter(CenterID)
);
INSERT INTO Fleet (FleetName, SupervisorName, CenterID) VALUES ('猛虎队', '张主管', 1);
```

4.1.3 创建 Vehicle 表并初始化

```sql
CREATE TABLE Vehicle (
    VehicleID INT PRIMARY KEY IDENTITY(1,1),
    PlateNumber NVARCHAR(20) NOT NULL UNIQUE,
    MaxWeight DECIMAL(10, 2) NOT NULL,
    MaxVolume DECIMAL(10, 2) NOT NULL,
    CurrentStatus NVARCHAR(20) DEFAULT '空闲' CHECK (CurrentStatus IN ('空闲', '运输中',
      '维修中', '异常')),
    FleetID INT NOT NULL,
    FOREIGN KEY (FleetID) REFERENCES Fleet(FleetID)
);
INSERT INTO Vehicle (PlateNumber, MaxWeight, MaxVolume, CurrentStatus, FleetID)
VALUES ('京A88888', 5000.00, 20.0, '空闲', 1),
       ('京B66666', 2000.00, 10.0, '运输中', 1);
```

4.1.4 创建 Driver 表并初始化

```sql
CREATE TABLE Driver (
    DriverID INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(50) NOT NULL,
    LicenseGrade NVARCHAR(10) CHECK (LicenseGrade IN ('A1', 'A2', 'B1', 'B2', 'C1')),
    Phone NVARCHAR(20),
    FleetID INT NOT NULL,
    FOREIGN KEY (FleetID) REFERENCES Fleet(FleetID)
);
INSERT INTO Driver (Name, LicenseGrade, Phone, FleetID)
VALUES ('李老四', 'A1', '13800138000', 1),
       ('王小五', 'B2', '13900139000', 1);
```

4.1.5 创建 Orders 表并初始化

```sql
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY IDENTITY(1,1),
    Weight DECIMAL(10, 2) NOT NULL,
    Volume DECIMAL(10, 2) NOT NULL,
    Destination NVARCHAR(100),
    Status NVARCHAR(20) DEFAULT '待分配' CHECK (Status IN ('待分配', '运输中', '已完成')
     ),
    VehicleID INT,
    DriverID INT NULL,  -- 新增：关联司机
    OrderTime DATETIME DEFAULT GETDATE(), -- 新增：下单时间
    FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID),
    FOREIGN KEY (DriverID) REFERENCES Driver(DriverID)
);
INSERT INTO Orders (Weight, Volume, Destination, Status, VehicleID, DriverID, OrderTime)
VALUES (1000.00, 5.0, '天津港', '已完成', 1, 1, DATEADD(day, -10, GETDATE())),
       (500.00, 2.0, '大兴机场', '运输中', 2, 2, DATEADD(day, -2, GETDATE()));
```

4.1.6 创建 Abnormality 表并初始化

```sql
CREATE TABLE Abnormality (
    AbnormalityID INT PRIMARY KEY IDENTITY(1,1),
    EventType NVARCHAR(20) NOT NULL,
    OccurTime DATETIME DEFAULT GETDATE(),
    FineAmount DECIMAL(10, 2) DEFAULT 0,
    IsHandled BIT DEFAULT 0,
    DriverID INT,
    VehicleID INT,
    FOREIGN KEY (DriverID) REFERENCES Driver(DriverID),
    FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID)
);
```

4.1.7 创建 History_Log 表

```sql
CREATE TABLE History_Log (
    LogID INT PRIMARY KEY IDENTITY(1,1),
    TableName NVARCHAR(50),
    RecordID INT,
    OperationType NVARCHAR(20),
    OldValue NVARCHAR(MAX),
    NewValue NVARCHAR(MAX),
    LogTime DATETIME DEFAULT GETDATE()
);
```

4.2 Trigger 的详细设计与代码实现

本系统设计了多种触发器以实现自动化运维和数据一致性，具体如下：

4.2.1 车辆分配后自动变更状态

设计意图：当运单分配车辆（即 Orders 表插入新记录且 VehicleID 非空）时，自动将对应车辆的状态变为“运输中”，确保车辆状态与业务流程同步。

```sql
CREATE TRIGGER trg_SetVehicleBusy
ON Orders
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @VehicleID INT;
    SELECT @VehicleID = VehicleID FROM inserted;
    UPDATE Vehicle
    SET CurrentStatus = '运输中'
    WHERE VehicleID = @VehicleID;
END;
```

4.2.2 运单签收后车辆自动闲置

设计意图：当运单状态变为“已完成”时，若该车辆无其他未完成运单，则自动将车辆状态变回“空闲”，实现车辆资源的自动释放。

```sql
CREATE TRIGGER trg_SetVehicleIdle
ON Orders
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @VehicleID INT;
    SELECT @VehicleID = VehicleID FROM inserted;
    IF NOT EXISTS(SELECT 1 FROM Orders WHERE VehicleID=@VehicleID AND Status <>'已完成')
    BEGIN
        UPDATE Vehicle SET CurrentStatus = '空闲' WHERE VehicleID = @VehicleID;
    END
END;
```

4.2.3 司机驾照变更自动审计

设计意图：当 Driver 表中的 LicenseGrade 字段被修改时，自动记录变更前后的驾照等级到审计日志表（History_Log），实现关键字段的变更追踪。

```
CREATE TRIGGER trg_DriverAudit
ON Driver
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF UPDATE(LicenseGrade)
    BEGIN
        INSERT INTO History_Log (TableName, RecordID, OperationType, OldValue, NewValue)
        SELECT
            'Driver',
            d.DriverID,
            'Update_License',
            d.LicenseGrade, -- 旧值
            i.LicenseGrade  -- 新值
        FROM deleted d
        INNER JOIN inserted i ON d.DriverID = i.DriverID;
    END
END;
```

4.3 存储过程与视图的设计

本系统充分利用存储过程（Stored Procedure）和视图（View）来提升数据操作的安全性、复用性和查询效率。以下结合教材规范，详细说明设计思路、实现代码及其作用。

一、存储过程设计 存储过程是事先经过编译并存储在数据库中的一组 SQL 语句，可实现批量操作、业务封装和权限隔离。以“统计某车队月度绩效”为例，设计如下：

- **设计目标：**统计指定车队在某年某月的总运单数、异常数、累计罚款。
- **输入参数：**车队 ID（@FleetID）、年份（@Year）、月份（@Month）。
- **输出结果：**车队名称、总运单数、异常总数、累计罚款金额。
- **适用场景：**车队绩效考核、月度报表自动生成等。

```
CREATE PROCEDURE sp_GetFleetMonthlyReport
    @FleetID INT,
    @Year INT,
    @Month INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT
        f.FleetName,
        COUNT(DISTINCT o.OrderID) AS TotalOrders,    -- 总运单数
        COUNT(DISTINCT a.AbnormalityID) AS TotalAbnormalities, -- 异常总数
        ISNULL(SUM(a.FineAmount), 0) AS TotalFines   -- 累计罚款
    FROM Fleet f
    LEFT JOIN Vehicle v ON f.FleetID = v.FleetID
    LEFT JOIN Orders o ON v.VehicleID = o.VehicleID
```

```
        AND YEAR(o.OrderTime) = @Year AND MONTH(o.OrderTime) = @Month
    LEFT JOIN Abnormality a ON v.VehicleID = a.VehicleID
        AND YEAR(a.OccurTime) = @Year AND MONTH(a.OccurTime) = @Month
    WHERE f.FleetID = @FleetID
    GROUP BY f.FleetName;
END;
```

Listing 1: 车队月度绩效统计存储过程

在 datagrip 中可以看见该存储过程：

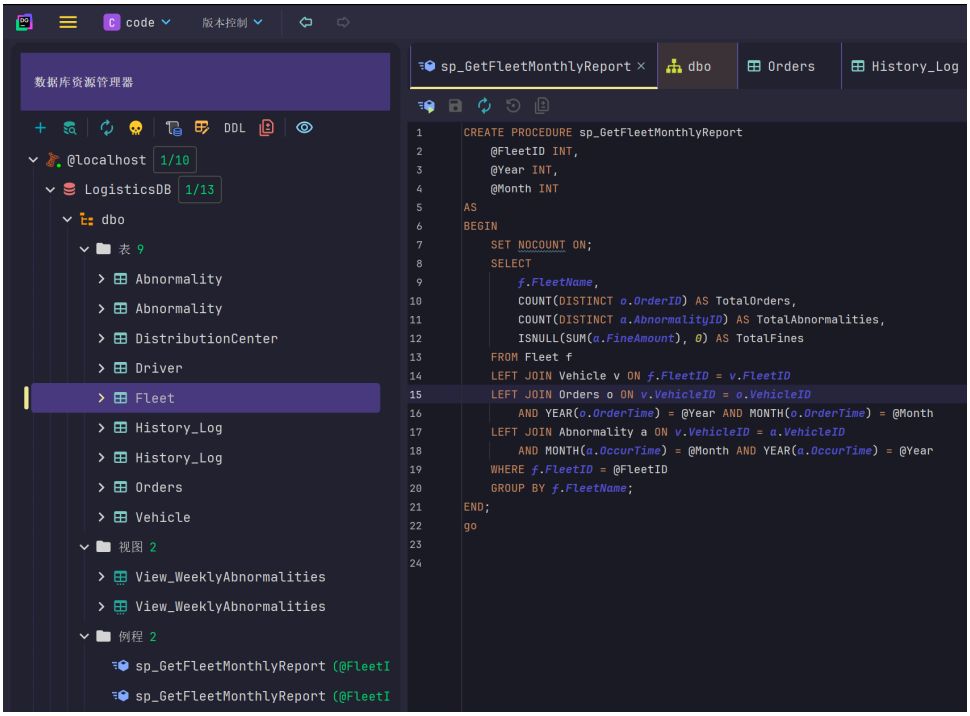


图 3: 存储过程在 DataGrip 中的展示

分析：

- 存储过程通过参数灵活指定统计范围，便于前端或报表系统调用。
- 采用多表连接，自动聚合车队下所有车辆的运单与异常数据。
- 业务逻辑集中于数据库端，提升安全性和性能，减少数据传输。
- 可扩展性强，后续如需增加统计项，仅需修改存储过程。

二、视图设计　视图是基于一条或多条查询语句的虚拟表，便于统一查询接口、简化复杂 SQL。以“本周异常事件视图”为例：

- **设计目标：**快速查询本周所有异常事件及其关联司机、车辆信息。
- **典型用途：**统计报表、异常监控、前端数据展示等。

```
CREATE VIEW View_WeeklyAbnormalities AS
SELECT a.AbnormalityID, a.EventType, a.OccurTime, d.Name AS DriverName, v.PlateNumber
FROM Abnormality a
JOIN Driver d ON a.DriverID = d.DriverID
```

```
5  JOIN Vehicle v ON a.VehicleID = v.VehicleID;
```

Listing 2: 本周异常事件视图

分析：

- 视图将多表关联逻辑封装，简化了前端和报表查询代码。
- 可直接用于权限控制，限制用户仅能访问视图数据。
- 支持与其他查询嵌套使用，提升 SQL 复用性。

4.4 索引的设置策略

在实际业务场景中，随着数据量的增加，数据库查询效率会显著受到影响。为提升高频字段的查询速度，本系统对关键字段设置了合适的索引，并通过 SQL Server 的“执行计划”工具直观展示了优化效果。

- **优化目标：**提高车牌号、运单日期、司机工号等高频查询字段的检索效率，减少全表扫描，降低 I/O 开销。
- **测试方法：**使用 SQL Server Management Studio (SSMS) 的“包含实际的执行计划”功能（Ctrl+M），对比建索引前后的查询计划变化。

案例一：车牌号索引 在测试前，我们模拟了 1000 条车辆数据，确保车牌号字段（`PlateNumber`）具有较高的查询频率。建索引前，执行 `SELECT * FROM Vehicle WHERE PlateNumber = '京A88888'`，执行计划显示为 **Table Scan**，即全表扫描。

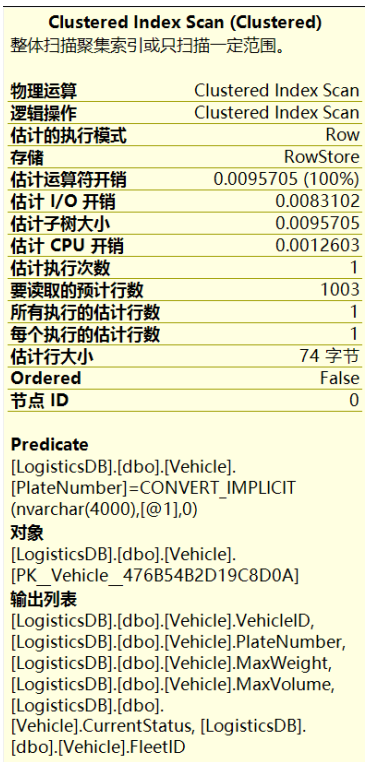


图 4: 车牌号查询执行计划（建索引前）

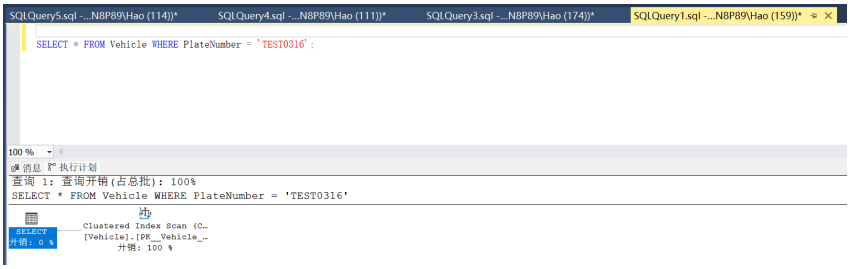


图 5: Table Scan 详细信息

建立索引：`CREATE INDEX IX_Vehicle_PlateNumber ON Vehicle(PlateNumber);`

建索引后，执行同样查询，执行计划变为 **Index Seek**，数据库可直接定位目标行，查询效率大幅提升。

Index Seek (NonClustered)
扫描非聚集索引中特定范围的行。

物理运算	Index Seek
逻辑操作	Index Seek
估计的执行模式	Row
存储	RowStore
估计运算符开销	0.0032831 (50%)
估计 I/O 开销	0.003125
估计子树大小	0.0032831
估计 CPU 开销	0.0001581
估计执行次数	1
要读取的预计行数	1
所有执行的估计行数	1
每个执行的估计行数	1
估计行大小	30 字节
Ordered	True
节点 ID	1

对象
[LogisticsDB].[dbo].[Vehicle].
[IX_Vehicle_PlateNumber]
输出列表
[LogisticsDB].[dbo].
[Vehicle].VehicleID, [LogisticsDB].
[dbo].[Vehicle].PlateNumber
Seek Predicates
Seek Keys[1]: 前缀: [LogisticsDB].[dbo].
[Vehicle].PlateNumber = 标量运算符
(CONVERT_IMPLICIT(nvarchar(4000),
[@1],0))

图 6: 车牌号查询执行计划（建索引后）

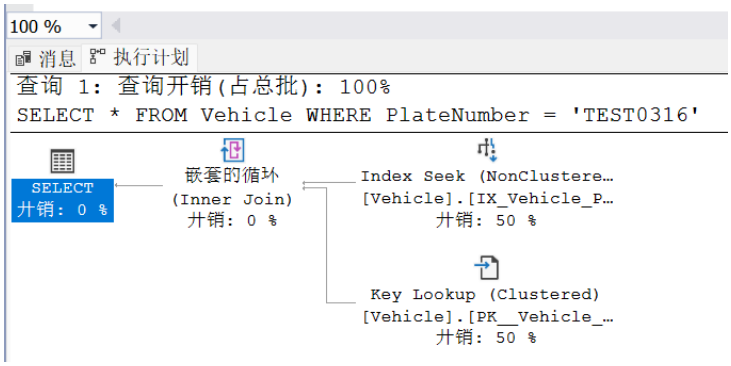


图 7: Index Seek 详细信息

实验现象与分析

- **未建索引时：**查询语句 `SELECT * FROM Vehicle WHERE PlateNumber = 'TEST0316'` 的执行计划为 **Clustered Index Scan**，即遍历整张 Vehicle 表。
  - 估计 I/O 开销为 0.008312。
  - 估计行数为 1003（即全表）。
  - 需要读取全部数据页，随着数据量增加，查询耗时显著上升。
- **建索引后：**对 PlateNumber 字段建立非聚集索引后，执行同样查询，执行计划变为 **Index Seek**，数据库可直接通过索引定位目标行。
  - 估计 I/O 开销降为 0.003125。
  - 估计只需读取 1 行数据。
  - 查询效率大幅提升，I/O 和 CPU 消耗显著降低。

结论：数据显示，未建索引时数据库需全表扫描，I/O 和 CPU 成本高；建索引后可直接定位目标数据，极大提升了查询性能。实际执行计划中的“估计 I/O 开销”“估计行数”等指标也直观反映了优化效果。因此，建议对高频查询字段建立合适索引，并结合业务场景定期优化索引结构。

5 系统实现与测试

5.1 开发环境说明

- **数据库管理系统：** Microsoft SQL Server 2019
- **开发语言：** Python 3.12
- **后端框架：** Flask 2.3
- **前端框架：** React 18

网页首页如图：



图 8: 系统首页

5.2 关键功能截图

5.2.1 录入信息后校验

操作：在“基础录入”界面输入错误的车牌信息，验证车牌校验触发器是否工作。



图 9: 录入车牌信息后触发校验

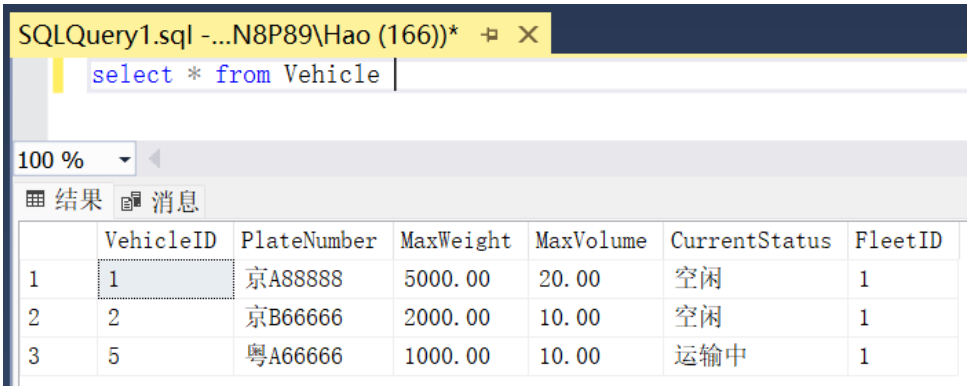
操作：分配车辆时触发器校验车辆是否超重超载。



图 10: 分配车辆时触发超重超载校验

5.2.2 分配车辆后状态变更

操作：在“运单管理”界面分配车辆后，验证车辆状态自动变更触发器是否工作。



	VehicleID	PlateNumber	MaxWeight	MaxVolume	CurrentStatus	FleetID
1	1	京A88888	5000.00	20.00	空闲	1
2	2	京B66666	2000.00	10.00	空闲	1
3	5	粤A66666	1000.00	10.00	运输中	1

图 11: 分配车辆后车辆状态变更

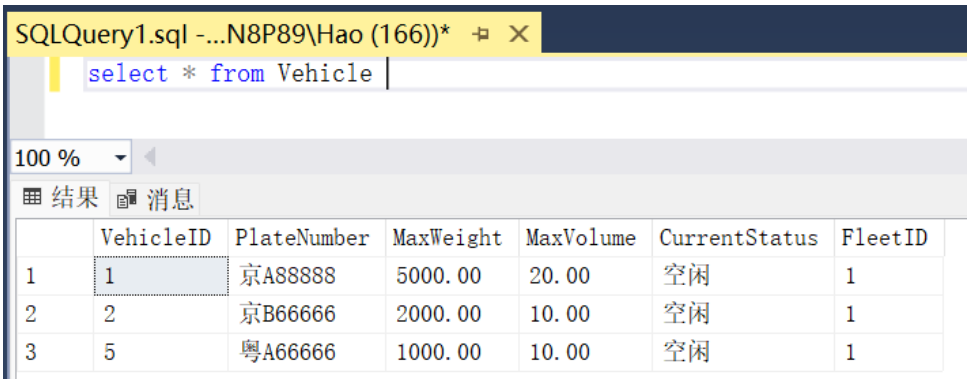
同时前端 UI 中将状态为运输中的车辆显示在运单签收界面，供用户选择签收。



图 12: 前端 UI 中车辆状态显示

5.2.3 运单签收后车辆闲置

操作：在“运单签收”界面签收运单后，验证车辆状态自动变更触发器是否工作。



	VehicleID	PlateNumber	MaxWeight	MaxVolume	CurrentStatus	FleetID
1	1	京A88888	5000.00	20.00	空闲	1
2	2	京B66666	2000.00	10.00	空闲	1
3	5	粤A66666	1000.00	10.00	空闲	1

图 13: 运单签收后车辆状态变更

同时前端 UI 中闲置的车辆不显示在运单签收界面。

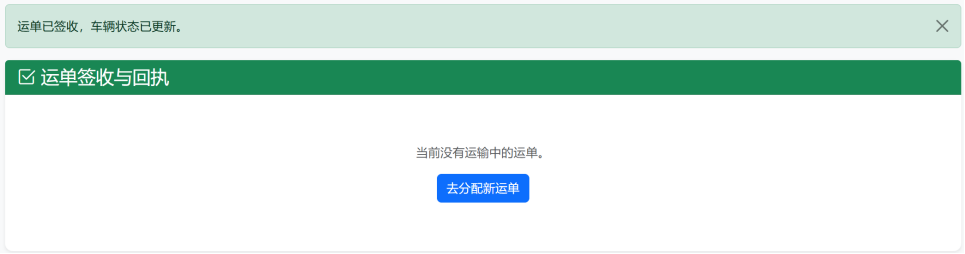
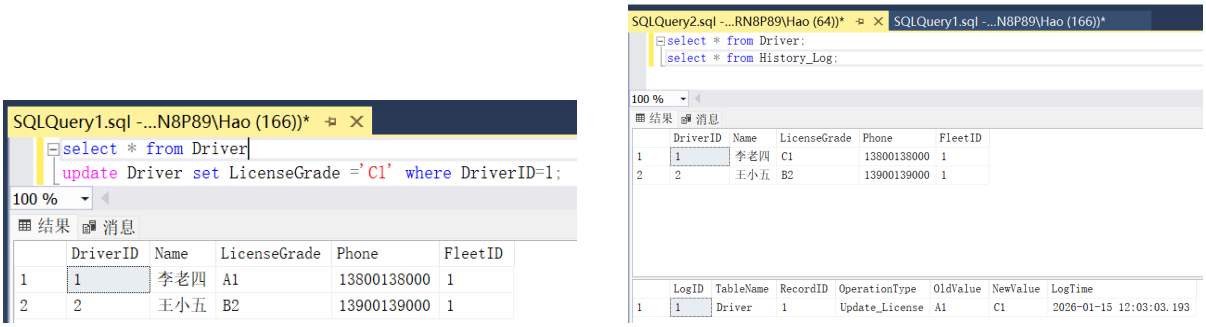


图 14: UI 中签收后车辆状态显示

5.2.4　司机驾照变更审计

操作：在 SSMS 中修改司机驾照等级，验证审计触发器是否工作。



(a) 修改前　　(b) 修改后及审计日志

图 15: 修改司机驾照等级触发审计

5.2.5　异常事件录入

操作：在“异常管理”界面录入异常事件，验证功能是否正常。



图 16: 异常事件录入界面

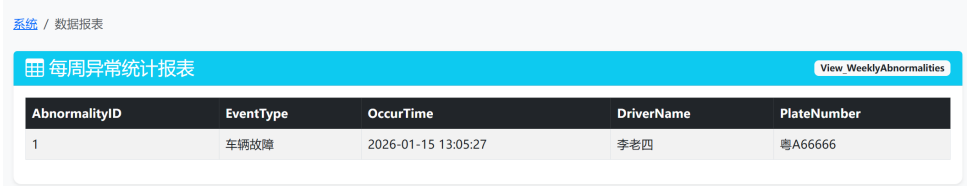


图 17: 异常事件录入结果

5.3　前端技术选型与关键代码

本系统前端采用 React 18 框架，结合 Ant Design 组件库实现高效开发。通过 axios 与后端 Flask API 进行数据交互，页面布局采用响应式设计，兼容主流浏览器。

主要技术选型如下：

- **React 18**：现代化前端框架，支持函数组件和 Hooks，提升开发效率。
- **Ant Design**：丰富的 UI 组件库，快速搭建美观界面。
- **axios**：轻量级 HTTP 客户端，负责与后端 API 通信。
- **ECharts**：数据可视化库，用于展示统计报表。

关键代码示例（录入表单）：

```
@app.route('/allocate', methods=['GET', 'POST'])
def allocate():
    db = get_db()
    if request.method == 'POST':
        try:
            weight = float(request.form['weight'])
            volume = float(request.form['volume'])
            destination = request.form['destination']
            vehicle_id = request.form['vehicle_id']

            db.create_order(weight, volume, destination, vehicle_id)
            flash('运单分配成功！', 'success')
            return redirect(url_for('allocate'))

        except pyodbc.ProgrammingError as e:
            # 专门捕获 SQL Server 的触发器错误 (RAISERROR)
            error_msg = str(e)
            # 提取 SQL 错误信息中的关键描述
            if "严重超载" in error_msg:
                flash('【操作拒绝】严重超载！分配导致超出车辆最大载重限制。', 'danger')
            else:
                flash(f'数据库错误: {error_msg}', 'danger')

        except pyodbc.Error as e:
            flash(f'系统错误: {e}', 'danger')
        except ValueError:
            flash('输入格式错误，重量和体积必须为数字。', 'danger')
        finally:
            db.close()

    # GET 请求
    idle_vehicles = db.get_idle_vehicles()
    db.close()
    return render_template('allocate.html', vehicles=idle_vehicles)

@app.route('/entry', methods=['GET', 'POST'])
def entry():
    db = get_db()
    if request.method == 'POST':
        form_type = request.form.get('form_type')

        if form_type == 'driver':
            name = request.form['name']
            license_grade = request.form['license_grade']
            phone = request.form['phone']
            fleet_id = request.form['fleet_id']

            if db.add_driver(name, license_grade, phone, fleet_id):
                flash('司机录入成功！', 'success')
            else:
```

```
                flash('司机录入失败，请检查数据。', 'danger')

        elif form_type == 'vehicle':
            plate = request.form['plate']
            try:
                max_weight = float(request.form['max_weight'])
                max_volume = float(request.form['max_volume'])
                fleet_id = request.form['fleet_id']

                if db.add_vehicle(plate, max_weight, max_volume, fleet_id):
                    flash('车辆录入成功！', 'success')
                else:
                    flash('车辆录入失败。', 'danger')
            except ValueError:
                flash('车辆载重和体积必须是数字。', 'danger')

        db.close()
        return redirect(url_for('entry'))

    # GET 请求：加载车队信息供下拉框使用
    fleets = db.get_fleets()
    db.close()
    return render_template('entry.html', fleets=fleets)

@app.route('/exception', methods=['GET', 'POST'])
def exception():
    db = get_db()
    if request.method == 'POST':
        try:
            vehicle_id = request.form['vehicle_id']
            driver_id = request.form['driver_id']
            event_type = request.form['event_type']
            occur_time = request.form['occur_time']
            fine_amount = float(request.form['fine_amount'])

            if db.add_abnormality(event_type, occur_time, fine_amount, driver_id,
 vehicle_id):
                flash('异常记录已保存。', 'success')
            else:
                flash('保存失败。', 'danger')
        except ValueError:
            flash('罚款金额格式错误。', 'danger')
        finally:
            db.close()
        return redirect(url_for('exception'))

    # GET 请求：加载车辆和司机列表
    vehicles = db.get_all_vehicles()
    drivers = db.get_all_drivers()
    current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    db.close()
    return render_template('exception.html', vehicles=vehicles, drivers=drivers,
 current_time=current_time)

@app.route('/report')
def report():
    db = get_db()
    columns, rows = db.get_weekly_abnormalities()
    db.close()
```

```
    return render_template('report.html', columns=columns, rows=rows)

# --- 修改部分 B: 添加 signoff 路由 (解决问题 2) ---
@app.route('/signoff', methods=['GET', 'POST'])
def signoff():
    db = get_db()
    if request.method == 'POST':
        # 点击了“确认签收”按钮
        order_id = request.form['order_id']
        if db.complete_order(order_id):
            flash('运单已签收，车辆状态已更新。', 'success')
        else:
            flash('签收失败，请重试。', 'danger')
        db.close()
        return redirect(url_for('signoff'))

    # GET 请求：显示当前活跃的运单
    active_orders = db.get_active_orders()
    db.close()
    return render_template('signoff.html', orders=active_orders)
```

5.4 业务逻辑扩展与实现

针对系统的高级管理需求，我们进一步实现了车队资源监控、司机绩效评估及月度安全报表功能。以下是核心业务逻辑的实现解析。

5.4.1 车队资源实时监控

该模块通过数据库聚合查询，实时展示某配送中心下所有车辆的负载状态（空闲、满载、运输中、异常），并动态计算当前载重/容积占用比率，辅助调度员进行精准决策。

```
@app.route('/fleet_resources')
def fleet_resources():
    db = get_db()
    center_id = request.args.get('center_id')
    status_filter = request.args.get('status', 'all')

    resources = []
    if center_id:
        # 调用后端逻辑：关联 Fleet, Vehicle, Orders 表
        # 并根据当前运单计算实时负载
        rows = db.get_fleet_resources(center_id)
        for row in rows:
            # 业务逻辑判断：根据当前载重、最大载重及车辆状态
            # 自动判定 '满载'、'空闲' 或 '运输中'
            # ... (具体状态机逻辑)
            pass

    return render_template('fleet_resources.html', resources=resources)
```

如图，我们查询计算机学院这个中心的车辆资源状态，辅助管理决策：



图 18: 车队资源实时监控界面

5.4.2 司机绩效多维度追踪

该功能支持按时间段（Start Date 至 End Date）查询特定司机的关键绩效指标，包括：总运输单量、产生异常事件次数及累计罚款金额。

为了支持维度分析，我们在后端实现了复杂的聚合查询，并对数据库模式进行了必要的迭代（参见附录部分）。

```
@app.route('/driver_performance')
def driver_performance():
    db = get_db()
    driver_id = request.args.get('driver_id')

    if driver_id:
        # 调用后端接口获取聚合数据
        # 1. 统计时间段内的订单总数
        # 2. 查询相关的异常记录详情
        # 3. 计算罚款总额
        order_count, abnormal_rows, total_fines = db.get_driver_performance(
            driver_id, start_time, end_time
        )

    return render_template('driver_performance.html',
                           order_count=order_count,
                           abnormal_rows=abnormal_rows,
                           total_fines=total_fines)
```

例如，我们查询我自己“欧阳易芃”在指定时间段内的绩效数据：

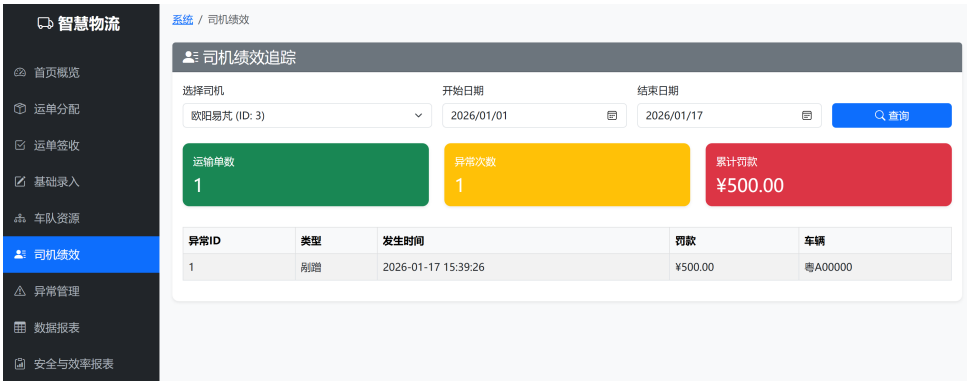


图 19: 司机绩效多维度追踪界面

5.4.3 车队月度安全与效率报表

通过调用存储过程 `sp_GetFleetMonthlyReport`，生成指定月份的车队综合报表，涵盖运单总量与异常统计，实现数据驱动的管理。

```
@app.route('/fleet_report')
def fleet_report():
    # ... 参数解析 ...
    if fleet_id:
        # 直接调用存储过程获取统计结果
        report_rows = db.get_fleet_monthly_report(int(fleet_id), int(year), int(month))
    return render_template('fleet_report.html', report_rows=report_rows)
```

例如，我们生成超算队在 26 年 1 月份的月度报表：



图 20: 车队月度安全与效率报表界面

5.4.4 前端交互优化与无障碍支持

为了提升系统的可用性与规范性，我们引入了 `static/styles.css` 进行统一的样式管理。同时，针对 HTML 模板进行了 Web 可访问性（Accessibility）优化：

- 为所有 `input` 和 `select` 表单元素添加了关联的 `label` 标签。
- 修复了部分下拉菜单缺少可访问名称的问题。
- 移除了内联样式，采用外部 CSS 类与 Bootstrap 结合的方式。

5.5 技术难点及解决方案

本项目在开发过程中遇到以下技术难点，并采取了相应解决方案：

- **前后端数据一致性**：由于物流业务流程复杂，需保证数据库状态与前端展示同步。解决方案：后端接口设计为幂等，所有状态变更均由数据库触发器自动维护，前端定时轮询或使用 WebSocket 实时刷新。
- **表单校验与异常处理**：车牌、载重等字段需严格校验。解决方案：前端使用 Ant Design 的表单校验，后端再做二次校验，数据库层通过触发器防止非法数据入库。
- **数据可视化**：统计报表需动态展示。解决方案：集成 ECharts，后端提供聚合接口，前端按需渲染图表。
- **性能优化**：数据量大时页面响应变慢。解决方案：分页加载、接口缓存、前端懒加载等。

6 总结

本次课程设计通过团队协作，完成了智慧物流系统的全流程开发。项目涵盖数据库建模、后端API、前端界面及自动化测试，锻炼了实际工程能力。

心得体会如下：

- **分工协作**：明确分工、定期沟通，有效提升开发效率。
- **技术选型**：合理选择主流框架和工具，降低开发难度。
- **效率优化**：通过触发器、存储过程等数据库高级对象，减少冗余操作，提高系统性能。
- **问题解决**：遇到技术难题时，查阅文档、团队讨论，快速定位并解决。

整体而言，本项目不仅提升了数据库与全栈开发能力，也加深了对团队协作和工程实践的理解。

7 附录/附件

7.1 演示 demo 及视频链接

上传到百度网盘，链接如下：**请助教/老师麻烦下载下来观看，在网盘上会很糊，谢谢!!**

> 通过网盘分享的文件：数据库大作业视频讲解.mp4
> 链接：https://pan.baidu.com/s/1Dd0tEnJr7tM6-0m_m8MeKQ?pwd=anqf 提取码：anqf

7.2 主要 SQL 脚本

本系统数据库初始化与修复脚本如下，完整原始内容如下：

7.2.1 一键初始化脚本（init_DB.sql）

```sql
USE master;
GO
IF EXISTS (SELECT * FROM sys.databases WHERE name = 'LogisticsDB')
BEGIN
    ALTER DATABASE LogisticsDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE LogisticsDB;
END
GO

CREATE DATABASE LogisticsDB;
GO
USE LogisticsDB;
GO

CREATE TABLE DistributionCenter (
    CenterID INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(50) NOT NULL,
    Location NVARCHAR(100)
);

CREATE TABLE Fleet (
    FleetID INT PRIMARY KEY IDENTITY(1,1),
    FleetName NVARCHAR(50) NOT NULL,
```

```
    SupervisorName NVARCHAR(50) NOT NULL,
    CenterID INT NOT NULL,
    FOREIGN KEY (CenterID) REFERENCES DistributionCenter(CenterID)
);

CREATE TABLE Vehicle (
    VehicleID INT PRIMARY KEY IDENTITY(1,1),
    PlateNumber NVARCHAR(20) NOT NULL UNIQUE,
    MaxWeight DECIMAL(10, 2) NOT NULL,
    MaxVolume DECIMAL(10, 2) NOT NULL,
    CurrentStatus NVARCHAR(20) DEFAULT '空闲' CHECK (CurrentStatus IN ('空闲', '运输中',
      '维修中', '异常')),
    FleetID INT NOT NULL,
    FOREIGN KEY (FleetID) REFERENCES Fleet(FleetID)
);

CREATE TABLE Driver (
    DriverID INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(50) NOT NULL,
    LicenseGrade NVARCHAR(10) CHECK (LicenseGrade IN ('A1', 'A2', 'B1', 'B2', 'C1')),
    Phone NVARCHAR(20),
    FleetID INT NOT NULL,
    FOREIGN KEY (FleetID) REFERENCES Fleet(FleetID)
);

CREATE TABLE Orders (
    OrderID INT PRIMARY KEY IDENTITY(1,1),
    Weight DECIMAL(10, 2) NOT NULL,
    Volume DECIMAL(10, 2) NOT NULL,
    Destination NVARCHAR(100),
    Status NVARCHAR(20) DEFAULT '待分配' CHECK (Status IN ('待分配', '运输中', '已完成')
     ),
    VehicleID INT,
    DriverID INT NULL,
    OrderTime DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID),
    FOREIGN KEY (DriverID) REFERENCES Driver(DriverID)
);

CREATE TABLE Abnormality (
    AbnormalityID INT PRIMARY KEY IDENTITY(1,1),
    EventType NVARCHAR(20) NOT NULL,
    OccurTime DATETIME DEFAULT GETDATE(),
    FineAmount DECIMAL(10, 2) DEFAULT 0,
    IsHandled BIT DEFAULT 0,
    DriverID INT,
    VehicleID INT,
    FOREIGN KEY (DriverID) REFERENCES Driver(DriverID),
    FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID)
);

CREATE TABLE History_Log (
    LogID INT PRIMARY KEY IDENTITY(1,1),
    TableName NVARCHAR(50),
    RecordID INT,
    OperationType NVARCHAR(20),
    OldValue NVARCHAR(MAX),
    NewValue NVARCHAR(MAX),
    LogTime DATETIME DEFAULT GETDATE()
```

```
);
GO

INSERT INTO DistributionCenter (Name, Location) VALUES ('中山大学计算机学院', '广州市番禺区');

INSERT INTO Fleet (FleetName, SupervisorName, CenterID) VALUES ('院楼车队', '张主管', 1);

INSERT INTO Vehicle (PlateNumber, MaxWeight, MaxVolume, CurrentStatus, FleetID)
VALUES ('粤A88888', 5000.00, 20.0, '空闲', 1),
       ('粤B66666', 2000.00, 10.0, '运输中', 1);

INSERT INTO Driver (Name, LicenseGrade, Phone, FleetID)
VALUES ('李老四', 'A1', '13800138000', 1),
       ('王小五', 'B2', '13900139000', 1);

INSERT INTO Orders (Weight, Volume, Destination, Status, VehicleID, DriverID, OrderTime)
VALUES (1000.00, 5.0, '香港', '已完成', 1, 1, DATEADD(day, -10, GETDATE())),
    (500.00, 2.0, '白云机场', '运输中', 2, 2, DATEADD(day, -2, GETDATE()));
GO

CREATE TRIGGER trg_CheckWeightLimit
ON Orders
AFTER INSERT, UPDATE
AS
BEGIN
    DECLARE @VehicleID INT;
    DECLARE @TotalWeight DECIMAL(10, 2);
    DECLARE @MaxWeight DECIMAL(10, 2);

    SELECT @VehicleID = VehicleID FROM inserted;
    IF @VehicleID IS NULL RETURN;

    SELECT @TotalWeight = SUM(Weight)
    FROM Orders
    WHERE VehicleID = @VehicleID AND Status <> '已完成';

    SELECT @MaxWeight = MaxWeight FROM Vehicle WHERE VehicleID = @VehicleID;

    IF ISNULL(@TotalWeight, 0) > @MaxWeight
    BEGIN
        RAISERROR ('【系统警告】严重超载！当前分配导致超出车辆最大载重，操作被拒绝。', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
GO

CREATE VIEW View_WeeklyAbnormalities AS
SELECT a.AbnormalityID, a.EventType, a.OccurTime, d.Name AS DriverName, v.PlateNumber
FROM Abnormality a
JOIN Driver d ON a.DriverID = d.DriverID
JOIN Vehicle v ON a.VehicleID = v.VehicleID;
GO

IF OBJECT_ID('sp_GetFleetMonthlyReport', 'P') IS NOT NULL DROP PROCEDURE sp_GetFleetMonthlyReport;
GO
```

```sql
CREATE PROCEDURE sp_GetFleetMonthlyReport
    @FleetID INT,
    @Year INT,
    @Month INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT
        f.FleetName,
        COUNT(DISTINCT o.OrderID) AS TotalOrders,
        COUNT(DISTINCT a.AbnormalityID) AS TotalAbnormalities,
        ISNULL(SUM(a.FineAmount), 0) AS TotalFines
    FROM Fleet f
    LEFT JOIN Vehicle v ON f.FleetID = v.FleetID
    LEFT JOIN Orders o ON v.VehicleID = o.VehicleID
        AND YEAR(o.OrderTime) = @Year AND MONTH(o.OrderTime) = @Month
    LEFT JOIN Abnormality a ON v.VehicleID = a.VehicleID
        AND YEAR(a.OccurTime) = @Year AND MONTH(a.OccurTime) = @Month
    WHERE f.FleetID = @FleetID
    GROUP BY f.FleetName;
END;
GO

PRINT '数据库 LogisticsDB 初始化完成！所有表、数据、触发器已就绪。';
```

7.2.2 修复与高级对象脚本（patch.sql）

为了支持业务逻辑扩展，本系统在后期迭代中引入了以下补丁脚本，涵盖存储过程、触发器及必要的表结构升级。

```sql
USE LogisticsDB;
GO

IF COL_LENGTH('Orders', 'DriverID') IS NULL
BEGIN
    ALTER TABLE Orders ADD DriverID INT NULL;
END;
GO

IF COL_LENGTH('Orders', 'OrderTime') IS NULL
BEGIN
    ALTER TABLE Orders ADD OrderTime DATETIME NULL;
END;
GO

IF COL_LENGTH('Orders', 'OrderTime') IS NOT NULL
BEGIN
    EXEC sp_executesql N'UPDATE Orders SET OrderTime = GETDATE() WHERE OrderTime IS NULL
      ';
END;
GO

IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Orders_Driver')
BEGIN
    ALTER TABLE Orders WITH NOCHECK
    ADD CONSTRAINT FK_Orders_Driver FOREIGN KEY (DriverID) REFERENCES Driver(DriverID);
END;
```

```
GO

IF OBJECT_ID('sp_GetFleetMonthlyReport', 'P') IS NOT NULL DROP PROCEDURE
    sp_GetFleetMonthlyReport;
GO
CREATE PROCEDURE sp_GetFleetMonthlyReport
    @FleetID INT,
    @Year INT,
    @Month INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT
        f.FleetName,
        COUNT(DISTINCT o.OrderID) AS TotalOrders,
        COUNT(DISTINCT a.AbnormalityID) AS TotalAbnormalities,
        ISNULL(SUM(a.FineAmount), 0) AS TotalFines
    FROM Fleet f
    LEFT JOIN Vehicle v ON f.FleetID = v.FleetID
    LEFT JOIN Orders o ON v.VehicleID = o.VehicleID
        AND YEAR(o.OrderTime) = @Year AND MONTH(o.OrderTime) = @Month
    LEFT JOIN Abnormality a ON v.VehicleID = a.VehicleID
        AND MONTH(a.OccurTime) = @Month AND YEAR(a.OccurTime) = @Year
    WHERE f.FleetID = @FleetID
    GROUP BY f.FleetName;
END;
GO

IF OBJECT_ID('trg_SetVehicleBusy', 'TR') IS NOT NULL DROP TRIGGER trg_SetVehicleBusy;
GO
CREATE TRIGGER trg_SetVehicleBusy
ON Orders
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @VehicleID INT;
    SELECT @VehicleID = VehicleID FROM inserted;

    UPDATE Vehicle
    SET CurrentStatus = '运输中'
    WHERE VehicleID = @VehicleID;
END;
GO

IF OBJECT_ID('trg_DriverAudit', 'TR') IS NOT NULL DROP TRIGGER trg_DriverAudit;
GO
CREATE TRIGGER trg_DriverAudit
ON Driver
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF UPDATE(LicenseGrade)
    BEGIN
        INSERT INTO History_Log (TableName, RecordID, OperationType, OldValue, NewValue)
        SELECT
            'Driver',
            d.DriverID,
```

```
            'Update_License',
            d.LicenseGrade,
            i.LicenseGrade
        FROM deleted d
        INNER JOIN inserted i ON d.DriverID = i.DriverID;
    END
END;
GO

IF OBJECT_ID('trg_SetVehicleIdle', 'TR') IS NOT NULL DROP TRIGGER trg_SetVehicleIdle;
GO
CREATE TRIGGER trg_SetVehicleIdle
ON Orders
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @VehicleID INT;
    SELECT @VehicleID = VehicleID FROM inserted;

    IF NOT EXISTS (SELECT 1 FROM Orders WHERE VehicleID = @VehicleID AND Status <> '已完成')
    BEGIN
        UPDATE Vehicle SET CurrentStatus = '空闲' WHERE VehicleID = @VehicleID;
    END
END;
GO
```